🛡️ 🚨 🔐

ПРОЄКТ 5 · DEFENCETECH · ~7 ГОД · ESP-IDF v5.5.4

Пульт озброєння системи охорони об’єкта (Arming Console)

🎯 МЕТА ПРОЄКТУ

01 · Крок 0 · Обладнання, схема, безпека ⏱ 30 хв

Схема складання на макетній платі ⏱ додатково 15 хв

02 · Крок 1 · Ініціалізація проєкту ESP-IDF ⏱ 25 хв

03 · Крок 2 · Модульна архітектура: перевикористання й новий компонент ⏱ 25 хв

04 · Крок 3 · C для MCU: типи станів системи безпеки ⏱ 25 хв

05 · Крок 4 · Сирена: зумер і реле ⏱ 30 хв

06 · Крок 5 · Кнопки: сире зчитування, дребезг на аналізаторі ⏱ 20 хв

07 · Крок 6 · PANIC як апаратне переривання ⏱ 45 хв

08 · Крок 7 · `esp_timer` : тік FSM і затримка з потенціометра ⏱ 40 хв

09 · Крок 8 · Повна FSM-дебаунс кнопки ⏱ 40 хв

10 · Крок 9 · Системна FSM: DISARMED → ARMING → ARMED → ALARM ⏱ 50 хв

11 · Крок 10 · Потенціометр → затримка й прискорення писку ⏱ 20 хв

12 · Крок 11 · Фінальна архітектура ⏱ 15 хв

13 · Крок 12 · Логи й діагностика ⏱ 15 хв

13.1 · Просте тестування: 4 кнопки й ручка

14 · Челенджі ⏱ ~1 год

Челендж 1 · Duress-код (тихий сигнал під примусом) ⏱ 20 хв

Челендж 2 · Третя й четверта зона ⏱ 15 хв

Челендж 3 · Лічильник хибних тривог і lockout ⏱ 15 хв

Челендж 4 · Індикація стану без LED ⏱ 10 хв

Челендж 5 · Збереження стану bypass у NVS ⏱ 20 хв

✅ ПІДСУМОК: ЩО МАЄТЕ ОТРИМАТИ В КІНЦІ

🛡️ 🚨 🔐

ПРОЄКТ 5 · DEFENCETECH · ~7 ГОД · ESP-IDF v5.5.4

Пульт озброєння системи охорони об’єкта (Arming Console)

> Покрокова практика: на ESP32-S3 (**чистий ESP-IDF, без Arduino**) будуємо навчальний тренажер пульта постановки об’єкта на охорону — той самий клас логіки, що керує реальними охоронними панелями периметра. Три кнопки-зони й одна кнопка централі працюють через **FSM-дебаунс** (Модуль 2.6), кнопка **ТРИВОГА (PANIC)** — через **справжнє апаратне переривання** (Модулі 2.3–2.4), бо сигнал лиха не має чекати на опитування. Над окремими FSM кнопок будуємо **системну FSM вищого рівня** (DISARMED → ARMING → ARMED → ALARM) — показуємо, як маленькі FSM компонуються у велику. Сирена (зумер + реле) — через транзисторний ключ BC547B (Модуль 2.2), а затримку постановки на охорону задає потенціометр, зчитуваний періодичним `esp_timer` (Модуль 2.5). Це виключно навчальний симулятор на макетній платі — жодних реальних систем безпеки чи озброєння не задіяно.

🧩 ESP-IDF з нуля · 🏗️ Модульні компоненти · ⚡ Апаратні переривання · ⏱️ esp_timer · 🎛️ Дворівнева FSM · 🏆 Челенджі

Інтегрує Модулі 2.1 (C++ для MCU), 2.2 (пасивні/активні навантаження), 2.3–2.4 (апаратні переривання), 2.5 (таймери), 2.6 (FSM-дебаунс).

🎯 МЕТА ПРОЄКТУ

У кінці ви отримаєте:

1. Надійний **ESP-IDF-фундамент**: модульні компоненти (`components/…`), а не файл-монстр — `load_driver` і `pot_sensor` спроєктовані так, що жодного слова “arm”/“zone”/“panic” в них немає, лише “GPIO → реле/сирена” і “АЦП → число” (доказ якості модульної архітектури).
2. Три кнопки-зони («ARM/DISARM», «ZONE 1», «ZONE 2»), кожна — окрема FSM-дебаунс (IDLE/PRESSED/HELD/RELEASE) із розрізненням короткого й довгого натиску.
3. Кнопку **PANIC**, підключену через **апаратне GPIO-переривання**, що вмикає тривогу миттєво й **незалежно від того, в якому стані зараз перебуває система** (озброєна, роззброєна, у процесі постановки) — ключова властивість реального duress/panic-входу в охоронних панелях.
4. **Другу, системну FSM** (`DISARMED → ARMING → ARMED → ALARM`), побудовану **над** FSM окремих кнопок — наочна демонстрація того, як прості автомати комбінуються у складнішу поведінку, не перетворюючись на “суп” з `if` .
5. Два незалежні `esp_timer` : 10-мс тік кнопкових FSM і 50-мс зчитування потенціометра, яким задається **затримка постановки на охорону** (entry/exit delay, 3–15 с) — з прискоренням писку зумера при наближенні кінця відліку.
6. Осцилограми з логічного аналізатора: порівняння часу реакції PANIC (переривання, мікросекунди) проти ZONE-кнопок (опитувана FSM, десятки мілісекунд) — та сама методична пара “переривання проти опитування”, але тепер у контексті “чому в охоронних системах duress-вхід ніколи не роблять через опитування”.

---

01 · Крок 0 · Обладнання, схема, безпека ⏱ 30 хв

Що на столі: ESP32-S3-DevKitC-1 N16R8 (USB-C, кабель **даних**), макетна плата, перемички, 2× **BC547B**, резистори **330 Ом** і **220 Ом**, реле **Songle** (синій модуль), потенціометр **B10K**, 4× тактові кнопки, зумер, логічний аналізатор **24 МГц / 8 каналів**.

> **🔴 ДВІ ОСОБЛИВОСТІ НАБОРУ — ТІ САМІ, ЩО В ПРОЄКТІ 4, ПОВТОРЮЄМО НАВМИСНО**
>
> 1. **Немає flyback-діода.** Так само вважаємо реле готовим **модулем** (драйвер, резистор і діод — на платі), керуємо напряму через `IN` , зовнішній транзистор на реле не потрібен. Якщо у вас “голе” реле без плати — обов’язково додайте зовнішній діод 1N4001/1N4148 паралельно котушці, перш ніж експлуатувати часто.
> 2. **Немає викрутки для гвинтових клем реле.** Контакти COM/NO/NC (де в реальній системі був би зовнішній сирена/стробоскоп на 12 В) залишаємо **невикористаними** — у цьому тренажері за звук і сигналізацію відповідає зумер, а реле лише демонструє факт комутації (клац) через свій керуючий 3-піновий роз’єм, який підключається звичайними дротами без викрутки.

```
ПІНАУТ ПРОЄКТУ (ESP32-S3, безпечні GPIO — уникаємо 0/3/19/20/26-32/39-46)

  GPIO4  ── вхід  (pull-down) ← BTN1  ARM / DISARM       (FSM, таймер-опитування)
  GPIO5  ── вхід  (pull-down) ← BTN2  ZONE 1 (тест зони)  (FSM, таймер-опитування)
  GPIO6  ── вхід  (pull-down) ← BTN3  ZONE 2 (тест зони)  (FSM, таймер-опитування)
  GPIO7  ── вхід  (pull-down) ← BTN4  PANIC / ТРИВОГА     (АПАРАТНЕ ПЕРЕРИВАННЯ!)
  GPIO10 ── вхід ADC1_CH9     ← потенціометр B10K — затримка постановки на охорону
  GPIO11 ── вихід             → база BC547B (220 Ом) → сирена (ЛИШЕ Варіант Б, Крок 4)
  GPIO12 ── вихід             → IN релейного модуля Songle — сигнальне реле,
                                  і за замовчуванням (Варіант А) ще й сирена через COM/NO

Це свідомий вибір: один перевірений апаратний "фундамент" — GPIO4-7 під кнопки з
FSM-дебаунсом, GPIO10 під АЦП, GPIO11-12 під силові навантаження — і своя бізнес-
логіка зверху (Крок 9). У реальній розробці продуктові лінійки будуються саме так.
```

Схема складання на макетній платі ⏱ додатково 15 хв

> Нижче — фізичне розташування деталей на стандартній макетній платі (830 точок, дві силові рейки зверху й знизу, центральний розрив між стовпчиками `a-e` і `f-j`). ESP32-S3-DevKitC-1 фізично завеликий, щоб ставити його в цю саму макетну плату поруч із усіма деталями — тримайте його **окремо** й ведіть до плати лише перемичками.



```
 8     .  .  .  .  .     .  .  .  .  .
 9     .  .  .  .  .     .  .  .  .  .   ← BTN3 (ZONE 2): ніжки a10/a12, f10/f12
10     .  .  .  .  .     .  .  .  .  .     перемичка f10→GPIO6, перемичка a12→ЛІВА рейка (+)
11     .  .  .  .  .     .  .  .  .  .
12     .  .  .  .  .     .  .  .  .  .   ← BTN4 (PANIC): ніжки a14/a16, f14/f16
13     .  .  .  .  .     .  .  .  .  .     перемичка f14→GPIO7, перемичка a16→ЛІВА рейка (+)
14     .  .  .  .  .     .  .  .  .  .
15     .  .  .  .  .     .  .  .  .  .
16     .  .  .  .  .     .  .  .  .  .
17     .  .  .  .  .     .  .  .  .  .   ← BC547B (сирена/зумер): 3 ніжки в c18, c19, c20
18     .  .  .  .  .     .  .  .  .  .     (пласка сторона — до себе; звірте E-B-C
19     .  .  .  .  .     .  .  .  .  .      мультиметром — розкладка залежить від виробника)
20     .  .  .  .  .     .  .  .  .  .
21     .  .  .  .  .     .  .  .  .  .   ← резистор 220 Ом: між f19 (база) і g19,
22     .  .  .  .  .     .  .  .  .  .     перемичка g19→GPIO11
23     .  .  .  .  .     .  .  .  .  .   ← перемичка e18 (емітер)→ЛІВА рейка (−/GND)
24     .  .  .  .  .     .  .  .  .  .   ← зумер "−"→e20 (колектор), зумер "+"→ЛІВА рейка (+)
25     .  .  .  .  .     .  .  .  .  .
26     .  .  .  .  .     .  .  .  .  .   ← потенціометр B10K (затримка постановки):
27     .  .  .  .  .     .  .  .  .  .     3 ніжки в c26, c27, c28
28     .  .  .  .  .     .  .  .  .  .   ← перемичка e26→ЛІВА рейка (+), e28→ЛІВА рейка (−)
29     .  .  .  .  .     .  .  .  .  .     перемичка f27→GPIO10 (повзунок/ADC)
       a  b  c  d  e     f  g  h  i  j
 +   ═══════════════════════════════════════════════════════════════
 −   ═══════════════════════════════════════════════════════════════

Релейний модуль Songle (сигнальне реле) — окремо від макетної плати (власна плата
з 3-піновим роз'ємом): VCC→ЛІВА рейка (+), GND→ЛІВА рейка (−), IN→GPIO12
напряму з ESP32, без макетної плати.
```

Порядок складання. У цьому проєкті є **два варіанти** підключення сирени — код (`components/load_driver`) підтримує обидва одним перемиканням макроса `LOAD_BUZZER_MODE` , тож обирайте те, що фізично зібрали, і виставте відповідний режим у Кроці 4:

- **Варіант А — сирена через контакти реле** (`LOAD_BUZZER_MODE_RELAY` , типове налаштування за замовчуванням): найпростіший монтаж, БЕЗ окремого транзистора для сирени — реле саме є вимикачем. Мінус — гучність не регулюється з коду, лише механічно (резистор послідовно, приглушення).
- **Варіант Б — сирена через транзисторний ключ** (`LOAD_BUZZER_MODE_TRANSISTOR`): на GPIO11 через BC547B (220 Ом), як типовий “зумер” у попередніх міні-проєктах курсу. Плюс — гучність можна керувати програмно через PWM (`load_buzzer_set_volume()`).

Виконуйте кроки 1-5 і 11-14 завжди; крок “сирена” — залежно від обраного варіанта.

1. Перемичкою з’єднайте `GND` ESP32-S3 із **синьою (−)** рейкою макетної плати.
2. Перемичкою з’єднайте `3V3` ESP32-S3 із **червоною (+)** рейкою макетної плати.
3. Мультиметром у режимі “прозвінка” переконайтесь, що між + і − рейками **немає** короткого замикання (опір — нескінченність/розрив), перш ніж підключати живлення. **Не пропускайте цей крок і крок 14** — у цьому проєкті помилково замкнена рейка живлення може одразу “розбудити” PANIC-переривання хибним сигналом ще до заливання коду.
4. Встановіть 4 кнопки (ARM, ZONE1, ZONE2, PANIC), кожна — **впоперек центрального розриву** плати (стандартний спосіб для 4-ногих тактових кнопок: розрив плати не дає їй “самозамкнутись”, поки не натиснеш).
5. Для кожної кнопки: одну ніжку (з боку `f`) з’єднайте перемичкою з відповідним GPIO (4/5/6/7), іншу ніжку (з боку `a` , через ряд нижче) — з **червоною (+)** рейкою.

Крок “сирена”, Варіант А (реле, за замовчуванням):

6А. Транзистор для сирени **не потрібен**. Сирена (звичайний баззер із двома виводами `+` / `−`) підключається напряму до вихідних клем релейного модуля: `+` живлення → `COM` , інший вивід сирени → `NO` (замикається, коли реле спрацьовує). Живлення сирени — окреме джерело (не 3.3V з ESP32), спільний `GND` з цим джерелом обов’язковий. 7А. (Опціонально, для регуляції гучності) резистор 100-330 Ом послідовно між `NO` і “+” сирени — трохи знижує гучність без переробки коду.

Крок “сирена”, Варіант Б (транзистор, для програмної гучності):

6Б. Встановіть **один** BC547B (для зумера) так, щоб 3 ніжки потрапили в 3 **окремі** сусідні ряди тієї самої колонки (`c` / `d` чи `f` / `g`) — кожен ряд плати є окремим електричним вузлом лише в межах 5 точок (`a-e` чи `f-j` окремо), тому 3 ніжки транзистора **обов’язково** мають бути в 3 різних рядах. 7Б. Мультиметром (режим діода) підтвердіть, яка ніжка — база, яка — колектор, яка — емітер (порядок **відрізняється** між виробниками BC547B — не покладайтесь сліпо на “стандартну” розкладку). 8Б. Резистор 220 Ом — між базою транзистора і окремим вільним рядом; перемичкою з’єднайте цей вільний ряд з GPIO11. 9Б. Емітер транзистора — перемичкою до **синьої (−)** рейки (спільний GND!). 10Б. Зумер: “+” до **червоної (+)** рейки, “−” до ряду з колектором транзистора.

Далі — спільні кроки, незалежно від варіанта:

11. Потенціометр B10K: крайні ніжки — до `+` / `−` рейок (будь-яка з двох, полярність ролі не грає, лише інвертує напрямок зростання значення), середня ніжка (повзунок) — перемичкою до GPIO10.

12. Релейний модуль Songle (окремо від макетної плати): `VCC` → `+`, `GND` → `−`, `IN` → GPIO12 напряму з ESP32. У Варіанті А сюди ж додатково йде сирена (пункт 6A); у Варіанті Б реле керує лише сигнальним контактом, без сирени.
13. Другий BC547B і резистор 330 Ом (якщо не використали для Варіанта Б) — відкладіть окремо як запасний комплект, потрібен лише якщо ваше реле виявиться “голим”, без власної плати.
14. Ще раз мультиметром прозвоніть `+` / `−` рейки на коротке замикання — і лише тоді підключайте USB.

🔵 **ЩО СПОСТЕРІГАЄМО** Якщо після складання й прошивки PANIC (BTN4) “стріляє” тривогою одразу після старту, без натискання — перевірте: (а) чи дійсно `pull_down_en` увімкнено в `gpio_config_t` для GPIO7, (б) чи не замкнута випадково перемичка `f14` одразу на `+` рейку повз кнопку.

🔴 **ТЕХНІКА БЕЗПЕКИ** • Перемикання — лише при **вийнятому USB**. Спільний GND для BC547B і сирени (Варіант Б) обов’язковий. Не більше 3.3 В на GPIO10 (АЦП). • **Це навчальний симулятор.** Жодних реальних сенсорів периметра, замків чи систем озброєння тут немає — лише кнопки, що імітують події (натиснув ZONE1 = “уявний датчик у зоні 1 спрацював”).

ПЛАН ПРАКТИКИ

Час	Крок	Що робимо
0:00–0:30	Крок 0	Схема, безпека
0:30–0:55	Крок 1	Ініціалізація ESP-IDF проєкту
0:55–1:20	Крок 2	Модульна архітектура + перевикористання `load_driver`
1:20–1:45	Крок 3	C для MCU: `enum class`-стиль станів системи (Модуль 2.1)
1:45–2:15	Крок 4	Сирена: реле, з опцією транзисторного ключа (Модуль 2.2)
2:15–2:40	Крок 5	Кнопки: сире зчитування, дребезг на аналізаторі
2:40–3:25	Крок 6	PANIC як апаратне переривання (Модулі 2.3–2.4)
3:25–4:05	Крок 7	`esp_timer`: тік FSM (10 мс) + затримка з потенціометра (50 мс)
4:05–4:50	Крок 8	Повна FSM-дебаунс кнопки (Модуль 2.6)
4:50–5:40	Крок 9	**Системна FSM**: DISARMED → ARMING → ARMED → ALARM
5:40–6:00	Крок 10	Потенціометр → затримка постановки й прискорення писку
6:00–6:20	Крок 11	Збірка `app_main.c`, фінальна архітектура
6:20–6:35	Крок 12	Логи й діагностика
6:35–7:10	Челенджі	5 завдань на розширення

---

02 · Крок 1 · Ініціалізація проєкту ESP-IDF ⏱ 25 хв

🟢 **МЕТА** Створити повністю самостійний ESP-IDF-проєкт `arming_console`. Цей документ не залежить від жодного іншого проєкту на диску — весь потрібний код компонентів наведено нижче в повному обсязі, нічого нізвідки копіювати не потрібно.

```
idf.py create-project arming_console
cd arming_console
idf.py set-target esp32s3
mkdir -p components/load_driver/include components/pot_sensor/include components/button_fsm/include
```

🟣 **РОЗБІР: чому саме так - `load_driver` і `pot_sensor` — це навмисно "тупі" апаратні драйвери**, які нічого не знають про кнопки чи бізнес-логіку конкретного застосунку, лише "GPIO → реле/зумер" і "АЦП → число". Такий компонент можна класти в будь-який проєкт без жодної правки — саме тому їхній повний код у Кроках 4 і 7 нижче ідентичний за духом тому, що міг би бути в іншому проєкті з такою самою периферією, і саме тому в них ви ніколи не побачите слів "arm", "zone", "panic". - `button_fsm` **пишемо з нуля, звіряючись із власним розумінням Модуля 2.6** (а не копіюючи чужий файл) — так матеріал краще закріплюється, і в цьому проєкті FSM кнопок отримує невелике розширення (PANIC-переривання, Крок 6).

🔵 **ЩО СПОСТЕРІГАЄМО** Порожні папки поки не дають `idf.py build` пройти — компоненти отримають вміст (`CMakeLists.txt` , `.h` , `.c`) у наступних кроках.

🟠 **МІНІ-ЗАВДАННЯ** Коли заповните `load_driver.h` у Кроці 4 — перевірте: чи є там хоч одне слово "arm", "zone", "panic"? Якщо немає — компонент спроєктовано правильно.

---

03 · Крок 2 · Модульна архітектура: перевикористання й новий компонент ⏱ 25 хв

🟢 **META** Структурувати проєкт на окремі **компоненти ESP-IDF**, а не тримати всю логіку в `main.c` — кожен компонент має свій `CMakeLists.txt` , публічний `.h` (в `include/`) і приватну реалізацію `.c` .

```
 arming_console/
├── CMakeLists.txt
├── main/
│   ├── CMakeLists.txt
│   └── app_main.c                ← "клей": системна логіка охорони
└── components/
    ├── load_driver/                ← Крок 4: реле + сирена (двома варіантами)
    │   ├── CMakeLists.txt
    │   ├── include/load_driver.h
    │   └── load_driver.c
    ├── button_fsm/                 ← Крок 6-8: кнопкова FSM + PANIC-переривання
    │   ├── CMakeLists.txt
    │   ├── include/button_fsm.h
    │   └── button_fsm.c
    └── pot_sensor/                 ← Крок 7: АЦП потенціометра на таймері
        ├── CMakeLists.txt
        ├── include/pot_sensor.h
        └── pot_sensor.c
```

`main/CMakeLists.txt` :

```
idf_component_register(
    SRCS "app_main.c"
    INCLUDE_DIRS "."
    REQUIRES button_fsm load_driver pot_sensor
)
```

🟣 **РОЗБІР: чому саме так - Системна FSM (DISARMED/ARMING/ARMED/ALARM) навмисно живе в `main/app_main.c`, а НЕ в компоненті `button_fsm`** — вона є **бізнес-логікою** конкретного продукту (охоронна панель), тоді як `button_fsm` — це загальний, перевикористовуваний механізм “натиснута кнопка → коротка/довга подія”, який годиться для будь-якого проєкту з подібними кнопками. Змішати їх в одному компоненті означало б, що наступного разу довелось би копіювати вже не чистий інструмент, а інструмент, “забруднений” логікою саме охоронної панелі. - Такий поділ — “тупі” перевикористовувані апаратні драйвери (`load_driver` , `pot_sensor`) окремо від бізнес-логіки конкретного продукту (`main.c`) — і є суттю “продуктової лінійки на спільному фундаменті”: той самий підхід годиться для будь-якого наступного проєкту з такою ж периферією.

🔵 **ЩО СПОСТЕРІГАЄМО** Порожні компоненти поки не компілюються повністю — заповнимо в наступних кроках.

🟠 **МІНІ-ЗАВДАННЯ** Якби вам знадобився третій проєкт (наприклад, “розумний парковочний шлагбаум”), які з трьох компонентів ви б узяли без змін, а які довелось би переписати?

---

04 · Крок 3 · C для MCU: типи станів системи безпеки ⏱ 25 хв

🟢 **META** Застосувати принципи Модуля 2.1 до опису стану охоронної системи — фіксовані `enum` , POD-структури, ніякого `malloc` , `volatile` для полів, що змінює ISR.

`main/app_main.c` — оголошення на початку файлу (типи системної FSM, Крок 9 — реалізація):

```c
#include <stdint.h>
#include <stdbool.h>

// Системна FSM — стан ВСІЄЇ охоронної панелі (Крок 9), не плутати з button_fsm_t
// (це стан ОДНІЄЇ кнопки, Модуль 2.6). Дві FSM різного рівня абстракції.
typedef enum {
    SYS_DISARMED = 0,   // система знята з охорони, тестування зон дозволене без тривоги
    SYS_ARMING,          // йде відлік затримки постановки (потенціометр задає тривалість)
    SYS_ARMED,           // на охороні: спрацювання будь-якої зони = тривога
    SYS_ALARM,           // тривога активна: сирена, доки не роззброять (BTN1 коротко)
} system_state_t;

// Один запис на кожну "зону" — теж POD, статичний масив, без динамічної пам'яті
typedef struct {
    const char *name;
    bool bypassed;      // виключена з контролю (довгий натиск відповідної кнопки, Крок 9)
    bool tripped;        // спрацювала (для логів/діагностики)
} zone_t;

#define N_ZONES 2
static zone_t s_zones[N_ZONES] = {
    { .name = "ZONE1", .bypassed = false, .tripped = false },
    { .name = "ZONE2", .bypassed = false, .tripped = false },
};

// volatile: пишеться з крос-контексту (ISR-задача PANIC, Крок 6), читається з main-циклу
static volatile bool s_panic_latched = false;
static system_state_t s_sys_state = SYS_DISARMED;
```

🟣 **РОЗБІР: чому саме так - Дві окремі FSM, які** не** треба зливати в одну велику** — `button_fsm_t` описує “коротко/довго натиснута ОДНА кнопка” і нічого не знає про “озброєно/роззброєно”. `system_state_t` описує стан **усієї** панелі і не знає, як саме детектується натискання (дребезг, таймери) — вона лише отримує вже “чисті” події від button_fsm. Це той самий принцип шарів (layers), що й у мережевих протоколах: кожен рівень знає лише про сусідній, не “заглядаючи” на два рівні вглиб. - **`zone_t` — масив, а не окремі глобальні змінні `zone1_bypassed` , `zone2_bypassed`** — легко додати `ZONE 3` , `ZONE 4` (Челендж), не переписуючи логіку, лише збільшивши `N_ZONES` і додавши рядок в ініціалізацію. - **`s_panic_latched` — volatile bool** — це прапорець, що виставляється з задачі, розбудженої PANIC-перериванням (Крок 6), і

читається з основного циклу системної FSM (Крок 9). `volatile` тут обов'язковий: без нього компілятор міг би оптимізувати повторне читання цього поля в циклі, "не помітивши", що воно змінюється ззовні (з іншого контексту виконання).

🔵 **ЩО СПОСТЕРІГАЄМО** Це чисто структурний крок — компіляція поки не дає видимого результату, це нормально, ми проєктуємо "скелет" перед логікою (Крок 9).

🟠 **МІНІ-ЗАВДАННЯ** Домалюйте на аркуші діаграму станів `system_state_t` із чотирма кружечками й підписаними стрілками переходів (поки що з пам'яті/здогаду) — звірите з Кроком 9.

---

05 · Крок 4 · Сирена: зумер і реле ⏱ 30 хв

🟢 **МЕТА** Реалізувати `load_driver` — компонент, що безпечно керує сигнальним реле й сиреною/зумером, **у двох підтримуваних варіантах підключення** (Крок 0): сирена через контакти реле, або сирена через окремий транзисторний ключ. Обирається одним макросом, решта коду проєкту (Крок 9) не залежить від вибору.

`components/load_driver/include/load_driver.h` (повністю):

```h
#pragma once
#include "driver/gpio.h"

#ifdef __cplusplus
extern "C" {
#endif

// Два підтримувані варіанти підключення сирени (Крок 0). Оберіть ОДИН і
// перезберіть — решта коду проєкту (main.c) не залежить від вибору.
//
//   LOAD_BUZZER_MODE_RELAY       сирена підключена до клем COM/NO реле,
//                                живиться від окремого джерела. Реле — єдиний
//                                вимикач, лише увімк/вимк, гучність з коду не
//                                регулюється.
//
//   LOAD_BUZZER_MODE_TRANSISTOR  сирена підключена напряму до LOAD_BUZZER_PIN
//                                через BC547B (резистор бази 220 Ом) — так,
//                                як типовий "зумер" у попередніх міні-проєктах.
//                                Керується через PWM (ledc), тому гучність
//                                можна міняти з коду (load_buzzer_set_volume()).
#define LOAD_BUZZER_MODE_RELAY       1
#define LOAD_BUZZER_MODE_TRANSISTOR  2

// <<< ВИСТАВТЕ ТУТ ФАКТИЧНУ СХЕМУ >>>
#define LOAD_BUZZER_MODE LOAD_BUZZER_MODE_RELAY

// База BC547B (220 Ом) — задіяний лише при LOAD_BUZZER_MODE_TRANSISTOR
#define LOAD_BUZZER_PIN GPIO_NUM_11
// IN релейного модуля — задіяний завжди; додатково керує сиреною через
// COM/NO при LOAD_BUZZER_MODE_RELAY
#define LOAD_RELAY_PIN GPIO_NUM_12

void load_driver_init(void);
void load_relay_set(bool on);
void load_buzzer_set(bool on);
void load_buzzer_chirp(uint32_t ms);

#if LOAD_BUZZER_MODE == LOAD_BUZZER_MODE_TRANSISTOR
// Гучність, 0..100 (%). Має ефект лише в режимі TRANSISTOR — реле є
// механічним вимикачем "увімк/вимк" і програмної регуляції гучності не має.
void load_buzzer_set_volume(uint8_t percent);
#endif
```

```
#ifdef __cplusplus
}
#endif
```

`components/load_driver/load_driver.c` **(повністю):**

```
#include <stdio.h>
#include "include/load_driver.h"
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

#if LOAD_BUZZER_MODE == LOAD_BUZZER_MODE_TRANSISTOR
#include "driver/ledc.h"
#endif

static const char *TAG = "load_driver";

// Багато готових релейних модулів (як-от Songle-плати із власним драйвером)
// насправді вмикають реле подачею LOW на IN, а не HIGH — попри те, що
// "логічно" очікуєш протилежне. Перевірте свій модуль (ручний тест: IN на
// GND напряму, в обхід ESP32) і виставте це значення відповідно — решта
// коду інвертувати не потрібно.
#define LOAD_RELAY_ACTIVE_LOW 1

#if LOAD_BUZZER_MODE == LOAD_BUZZER_MODE_TRANSISTOR
#define BUZZER_LEDC_TIMER    LEDC_TIMER_0
#define BUZZER_LEDC_MODE     LEDC_LOW_SPEED_MODE
#define BUZZER_LEDC_CHANNEL  LEDC_CHANNEL_0
#define BUZZER_LEDC_FREQ_HZ  2000                // чутний тон для п'єзо-зумера
#define BUZZER_LEDC_RES      LEDC_TIMER_8_BIT      // 0..255 кроків шпаруватості -> гучність

static uint8_t s_buzzer_volume_percent = 60;       // гучність за замовчуванням
#endif

void load_driver_init(void) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << LOAD_RELAY_PIN),   // буzzer-пін у режимі RELAY взагалі не потрібен;
                                                      // у режимі TRANSISTOR він конфігурується нижче через ledc, не тут
        .mode         = GPIO_MODE_OUTPUT,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&cfg);
    load_relay_set(false);   // через абстракцію, а не сирий рівень — тоді ACTIVE_LOW враховується і тут

#if LOAD_BUZZER_MODE == LOAD_BUZZER_MODE_TRANSISTOR
    ledc_timer_config_t timer_cfg = {
        .speed_mode      = BUZZER_LEDC_MODE,
        .duty_resolution = BUZZER_LEDC_RES,
        .timer_num       = BUZZER_LEDC_TIMER,
        .freq_hz         = BUZZER_LEDC_FREQ_HZ,
        .clk_cfg         = LEDC_AUTO_CLK,
    };
    ledc_timer_config(&timer_cfg);

    ledc_channel_config_t channel_cfg = {
        .gpio_num   = LOAD_BUZZER_PIN,
        .speed_mode = BUZZER_LEDC_MODE,
        .channel    = BUZZER_LEDC_CHANNEL,
        .timer_sel  = BUZZER_LEDC_TIMER,
        .duty       = 0,
        .hpoint     = 0,
    };
    ledc_channel_config(&channel_cfg);
#endif
}
```

```
void load_relay_set(bool on) {
#if LOAD_RELAY_ACTIVE_LOW
    int level = on ? 0 : 1;
#else
    int level = on ? 1 : 0;
#endif
    gpio_set_level(LOAD_RELAY_PIN, level);
    ESP_LOGI(TAG, "RELAY GPIO%d -> %d (logical %s)", LOAD_RELAY_PIN, level, on ? "ON" : "OFF");
}

#if LOAD_BUZZER_MODE == LOAD_BUZZER_MODE_RELAY

// Сирена фізично комутується контактами реле — просто передоручаємо
// load_relay_set(). Окрема функція лишається, щоб решта коду (Крок 9,
// підтвердження, наростаючий писк) не залежала від того, який режим обрано.
void load_buzzer_set(bool on) {
    load_relay_set(on);
}

#else // LOAD_BUZZER_MODE_TRANSISTOR

void load_buzzer_set_volume(uint8_t percent) {
    if (percent > 100) percent = 100;
    s_buzzer_volume_percent = percent;
}

void load_buzzer_set(bool on) {
    uint32_t max_duty = (1u << BUZZER_LEDC_RES) - 1u;                    // 255 для 8-бітної шпаруватості
    uint32_t duty = on ? (max_duty * s_buzzer_volume_percent) / 100u : 0u;
    ledc_set_duty(BUZZER_LEDC_MODE, BUZZER_LEDC_CHANNEL, duty);
    ledc_update_duty(BUZZER_LEDC_MODE, BUZZER_LEDC_CHANNEL);
    ESP_LOGI(TAG, "BUZZER GPIO%d PWM duty=%lu/%lu (volume=%u%%, %s)",
             LOAD_BUZZER_PIN, (unsigned long)duty, (unsigned long)max_duty,
             s_buzzer_volume_percent, on ? "ON" : "OFF");
}

#endif

void load_buzzer_chirp(uint32_t ms) {
    // Короткий блокуючий сигнал ДОПУСТИМИЙ лише для миттєвого "біп" (10-200 мс)
    // із контексту звичайної задачі (не ISR, не таймерний колбек) — Модуль 2.2/2.5.
    load_buzzer_set(true);
    vTaskDelay(pdMS_TO_TICKS(ms));
    load_buzzer_set(false);
}
```

`components/load_driver/CMakeLists.txt` :

```
idf_component_register(
    SRCS "load_driver.c"
    INCLUDE_DIRS "include"
    REQUIRES
        driver
        esp_driver_ledc
)
```

🟣 **РОЗБІР: чому саме так - `LOAD_BUZZER_MODE` — один макрос, дві реалізації `load_buzzer_set()`** — обирається на етапі компіляції (`#if`), тому жоден виклик у Кроці 9 (`load_buzzer_set()` , `load_buzzer_chirp()`) не потребує зміни залежно від того, яку схему ви фізично зібрали. Це той самий принцип інкапсуляції з Кроку 2 — застосунок працює з абстракцією “сирена увімкнена/вимкнена”, а не з конкретним GPIO чи транзистором. - `LOAD_RELAY_ACTIVE_LOW` — на практиці багато готових релейних модулів (як-от типова “синя” Songle-плата) вмикають реле подачею **LOW**, а не HIGH, на `IN` , попри інтуїтивне очікування протилежного. Перевіряється ручним тестом (`IN` на `GND` напряму, в обхід ESP32, з окремим живленням `VCC` / `GND` модуля): якщо клацає саме при `IN=GND` — тут `1` . - **`load_driver_init()` вимикає реле через `load_relay_set(false)` , а не через сирий `gpio_set_level(pin, 0)`** — здавалося б, дрібниця, але якщо писати рівень напряму в обхід абстракції, `LOAD_RELAY_ACTIVE_LOW` перестає діяти саме в момент старту плати: пін завжди піде в `0` , і якщо модуль активний по LOW — реле (і, у Варіанті А, сирена) увімкнеться **одразу при старті**, ще до першого натискання будь-якої кнопки. Такий клас помилок (“одна забута гілка коду не пройшла крізь ту саму абстракцію, що й решта”) — типова причина “чому воно поводиться інакше, ніж я задумав, хоча логіка на вигляд

правильна”. - **PWM (`ledc`) у Варіанті Б** — на відміну від простого `gpio_set_level` , `ledc` дозволяє задати **шпаруватість** (яку частку часу пін проведе в HIGH) — для активного зумера це напряму впливає на гучність: менша шпаруватість = менша середня напруга на зумері = тихіше. У Варіанті А (реле) такого важеля немає в принципі — реле або замкнене, або ні, третього не дано.

🔵 **ЩО СПОСТЕРІГАЄМО** `load_buzzer_chirp(100)` дає чутний сигнал (клацання реле у Варіанті А, “біп” зумера у Варіанті Б); лог `load_driver` показує кожну зміну — `RELAY GPIO12 -> ...` або `BUZZER GPIO... PWM duty=...` .

🟠 **МІНІ-ЗАВДАННЯ** Виміряйте аналізатором затримку між командою і реальним клацанням/звуком — механічне реле завжди на кілька мілісекунд повільніше електронного транзисторного ключа.

🔊 **РЕГУЛЮВАННЯ ГУЧНОСТІ СИРЕНИ** - **Варіант Б (транзистор)** — справжня програмна регуляція: `load_buzzer_set_volume(percent)` міняє шпаруватість PWM, а отже й гучність, без жодних апаратних змін. - **Варіант А (реле)** — програмно гучність не регулюється (реле — вимикач, не регулятор). Апаратні варіанти: (1) резистор 100-330 Ом послідовно між `NO` реле і “+” сирени (просте зниження гучності, підбирається дослідно, при завеликому опорі сирена може взагалі замовкнути); (2) фізичне приглушення (скотч/поролон на отворі сирени) — грубо, зате легко регулюється на слух і завжди можна прибрати.

---

06 · Крок 5 · Кнопки: сире зчитування, дребезг на аналізаторі ⏱ 20 хв

🟢 **META** Переконатись “живцем”, що без дебаунсу сигнал кнопки непридатний для логіки “озброєно/роззброєно”, де хибне спрацювання неприпустиме.

Тимчасовий тестовий код (буде замінений у Кроці 8) — `main/main.c` :

```c
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

#define BTN_ARM_PIN GPIO_NUM_4

static const char *TAG = "raw_read";

void app_main(void) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << BTN_ARM_PIN),
        .mode         = GPIO_MODE_INPUT,
        .pull_down_en = GPIO_PULLDOWN_ENABLE,   // внутрішня підтяжка ВНИЗ, ESP32-S3
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&cfg);

    while (1) {
        ESP_LOGI(TAG, "ARM = %d", gpio_get_level(BTN_ARM_PIN));
        vTaskDelay(pdMS_TO_TICKS(5));   // навмисно швидко — щоб зловити дребезг у логах
    }
}
```

🟣 **РОЗБІР: чому саме так** - `GPIO_PULLDOWN_ENABLE` — без нього “відпущений” пін висить у повітрі (floating) і ловить наведення; натиснуто = HIGH, не натиснуто = LOW. - `gpio_get_level()` **повертає** `0` / `1` **миттєво, без фільтрації** — саме тому під час фізичного натискання в моніторі видно **пачку** швидких переходів 0↔1, перш ніж рівень стабілізується — це і є дребезг контактів. - У системі безпеки хибне подвійне спрацювання ARM-кнопки через дребезг — не просто незручність (як зайве блимання LED), а **функційна помилка**: два швидкі перемикання поспіль означали б

“озброїли і одразу роззброїли” — тобто система лишилась би без охорони, хоча користувач думає, що озброїв її. Це робить якісний дебаунс (Крок 8) не просто гарною практикою, а вимогою безпеки.

🔵 **ЩО СПОСТЕРІГАЄМО** Дребезг контактів 2-15 мс на фронті — природа механічних кнопок, однакова незалежно від застосунку чи проєкту.

🟠 **МІНІ-ЗАВДАННЯ** Порахуйте: скільки хибних `SYS_ARMED ↔ SYS_DISARMED` перемикань дав би необроблений дребезг на один дотик, якби ви напряму викликали `toggle()` у `gpio_get_level()` без жодної FSM? (Подивіться кількість переходів на аналізаторі за один клік.)

---

07 · Крок 6 · PANIC як апаратне переривання ⏱ 45 хв

🟢 **МЕТА** Реалізувати кнопку тривоги так, щоб вона спрацьовувала **миттєво й завжди**, незалежно від поточного стану системної FSM (навіть якщо система в процесі `SYS_ARMING` чи вже `SYS_ALARM`) — це стандартна вимога до duress/panic-входу в реальних охоронних панелях: він не повинен залежати від “нормального” потоку керування.

`components/button_fsm/include/button_fsm.h` **(додаток):**

```
#define PANIC_PIN   GPIO_NUM_7

void panic_sensor_init(void (*on_panic_deferred)(void));
```

`components/button_fsm/button_fsm.c` **(додаток):**

```
#include "freertos/queue.h"
#include "esp_log.h"

static const char *TAG = "panic_isr";
static QueueHandle_t s_panic_queue = NULL;

static void IRAM_ATTR panic_isr_handler(void *arg) {
    uint32_t dummy = 1;
    BaseType_t woken = pdFALSE;
    xQueueSendFromISR(s_panic_queue, &dummy, &woken);
    if (woken) portYIELD_FROM_ISR();
}

static void panic_task(void *arg) {
    void (*callback)(void) = (void (*)(void))arg;
    uint32_t dummy;
    while (1) {
        if (xQueueReceive(s_panic_queue, &dummy, portMAX_DELAY)) {
            ESP_LOGE(TAG, "!!! PANIC — апаратне переривання GPIO%d !!!", PANIC_PIN);
            callback();
        }
    }
}

void panic_sensor_init(void (*on_panic_deferred)(void)) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << PANIC_PIN),
        .mode         = GPIO_MODE_INPUT,
        .pull_down_en = GPIO_PULLDOWN_ENABLE,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .intr_type    = GPIO_INTR_POSEDGE,
    };
    gpio_config(&cfg);

    s_panic_queue = xQueueCreate(4, sizeof(uint32_t));
```

```
    gpio_install_isr_service(0);
    gpio_isr_handler_add(PANIC_PIN, panic_isr_handler, NULL);

    // Пріоритет 15 — НАЙВИЩИЙ у системі: тривога має "перебити" навіть FSM-тік і АЦП
    xTaskCreate(panic_task, "panic_task", 3072, (void *)on_panic_deferred, 15, NULL);
}
```

🟣 **РОЗБІР: чому саме так** - **Патерн ISR→черга→задача** — і це навмисно: у вбудованих системах безпекові/критичні до часу події (panic-кнопка, аварійна зупинка) **завжди** обробляються однією й тією ж перевіреною схемою. Уніфікований підхід зменшує кількість місць, де можна припуститись помилки. - **ISR (`panic_isr_handler`) робить лише мінімум —** `xQueueSendFromISR` — переривання виконується з найвищим пріоритетом, перериваючи все інше, включно з ядром FreeRTOS. Будь-яка повільна операція в ньому (лог, доступ до флеша) заблокувала б систему на непередбачуваний час. Правило: **ISR лише сигналізує, задача — обробляє**. - **Пріоритет задачі** `panic_task` **— 15, найвищий у системі** — вище навіть за таймерний тік FSM-кнопок і задачі АЦП. Свідомий вибір: на панелі охорони тривога користувача повинна “обігнати” навіть звичайну логіку постановки на охорону, що може саме виконуватись. - **Незалежність від** `system_state_t` — зверніть увагу: `panic_sensor_init()` нічого не знає про `SYS_ARMED` / `SYS_DISARMED` . Callback (`on_panic_deferred` , Крок 9) сам вирішує, що робити, і це рішення буде: “тривога вмикається ЗАВЖДИ, в будь-якому стані” — саме так, як має поводитись реальний duress-вхід.

🔵 **ЩО СПОСТЕРІГАЄМО** Натисніть PANIC у будь-якому стані системи (навіть SYS_DISARMED, коли, здавалось б, “тривозі немось спрацьовувати”) — сирена вмикається миттєво. На аналізаторі затримка між фронтом на GPIO7 і реакцією на GPIO11/12 — одиниці мілісекунд, незалежно від навантаження на систему.

🟠 **МІНІ-ЗАВДАННЯ** Штучно “завантажте” систему (наприклад, тимчасовий `for` на мільйон ітерацій у `app_main` без `vTaskDelay`) і перевірте: чи PANIC все одно спрацьовує миттєво? Поясніть, чому апаратне переривання не залежить від того, наскільки “зайнятий” основний код (підказка: воно перериває CPU на апаратному рівні, а не чекає своєї черги в планувальнику).

🔴 **ПОШИРЕНА АПАРАТНА ПРОБЛЕМА: PANIC “стріляє” сам, без натискання** На макетній платі GPIO7 — найдовший провідний шлях до кнопки, і саме він найчастіше ловить наведення. Симптоми: у логах з’являються серії `!!! PANIC — апаратне переривання GPIO7 !!!` без фізичного натискання, або **кілька** спрацювань `panic_isr` за десятки мілісекунд на **один** дотик (реальний дребезг контакту, який `GPIO_INTR_POSEDGE` сам по собі не фільтрує — переривання реагує на кожен окремий фронт LOW→HIGH, а дребезг — це якраз кілька таких фронтів поспіль).

Перевірка методом виключення: прошийте плату і **нічого не натискайте** секунд 15-20, дивлячись у монітор. Якщо `panic_isr` спрацьовує сам — це підтверджений апаратний шум, а не помилка коду.

Виправлення — **зовнішній резистор 10 кОм між GPIO7 і GND**, паралельно до вбудованої підтяжки чіпа (~45 кОм). Обидва тягнуть пін в один бік (GND), не конфліктують, а зовнішній, маючи менший опір, “перебиває” наведення й дає значно чистіший сигнал. Той самий фікс варто зробити і на GPIO4/5/6 (кнопки ARM/ZONE1/ZONE2), якщо спостерігаєте на них аналогічні хибні спрацювання (наприклад, система сама “заходить” в `ARMING` / `DISARMED` без натискань).

---

08 · Крок 7 · `esp_timer` : тік FSM і затримка з потенціометра ⏱ 40 хв

🟢 **МЕТА** Один `esp_timer` на 10 мс тикає кнопкові FSM, другий на 50 мс читає потенціометр — тут це значення інтерпретується як **тривалість затримки постановки на охорону** (3–15 с), а не як швидкість блимання.

`components/pot_sensor/include/pot_sensor.h` :

```
#pragma once
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

void pot_sensor_init(void);          // старт періодичного esp_timer на 50 мс
uint16_t pot_sensor_get_raw(void);   // останнє прочитане значення 0..4095 (кеш, без блокувань)
```

```
#ifdef __cplusplus
}
#endif
```

`components/pot_sensor/pot_sensor.c`:

```
#include "pot_sensor.h"
#include "esp_timer.h"
#include "esp_adc/adc_oneshot.h"

#define POT_ADC_UNIT     ADC_UNIT_1
#define POT_ADC_CHANNEL  ADC_CHANNEL_9   // GPIO10 на ESP32-S3 = ADC1_CH9
#define POT_SAMPLE_MS    50u

static adc_oneshot_unit_handle_t s_adc_handle;
static volatile uint16_t s_last_raw = 0;   // volatile: пише таймер, читає main/pot_sensor_get_raw

static void pot_timer_cb(void *arg) {
    int raw = 0;
    adc_oneshot_read(s_adc_handle, POT_ADC_CHANNEL, &raw);
    s_last_raw = (uint16_t)raw;
}

void pot_sensor_init(void) {
    adc_oneshot_unit_init_cfg_t unit_cfg = { .unit_id = POT_ADC_UNIT };
    adc_oneshot_new_unit(&unit_cfg, &s_adc_handle);

    adc_oneshot_chan_cfg_t chan_cfg = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,   // 12 біт на ESP32-S3 → 0..4095
        .atten    = ADC_ATTEN_DB_12,        // дозволяє міряти до ~3.3В (замість дефолтних ~1.1В)
    };
    adc_oneshot_config_channel(s_adc_handle, POT_ADC_CHANNEL, &chan_cfg);

    const esp_timer_create_args_t args = {
        .callback = &pot_timer_cb,
        .name     = "pot_sample",
    };
    esp_timer_handle_t timer;
    esp_timer_create(&args, &timer);
    esp_timer_start_periodic(timer, POT_SAMPLE_MS * 1000ULL);   // esp_timer рахує в МІКРОсекундах!
}

uint16_t pot_sensor_get_raw(void) {
    return s_last_raw;   // volatile uint16_t читається атомарно на 32-бітному ядрі — гонки даних немає
}
```

`components/pot_sensor/CMakeLists.txt`:

```
idf_component_register(
    SRCS "pot_sensor.c"
    INCLUDE_DIRS "include"
    REQUIRES
        esp_adc
        esp_timer
)
```

🟣 **РОЗБІР: чому саме так - `esp_timer` рахує в мікросекундах** — `esp_timer_start_periodic(handle, period_us)`. Найпоширеніша помилка новачків: забути домножити мілісекунди на 1000. - `ADC_ATTEN_DB_12` — без цього опорна напруга ~1.1В, і потенціометр на 3.3В “впреться” в максимум задовго до кінця повороту ручки. - `volatile uint16_t s_last_raw` — пишеться в контексті таймерної задачі, читається з іншої задачі. `volatile` не дає компілятору закешувати значення в регістрі й “забути” перечитати його з пам’яті.

Таймер FSM-тіка — `components/button_fsm/button_fsm.c`:

```c
static void fsm_timer_cb(void *arg) {
    uint32_t now_ms = (uint32_t)(esp_timer_get_time() / 1000ULL);
    app_fsm_tick_all(now_ms);   // реалізація — Крок 9 цього документа
}

void button_fsm_start_timer(void) {
    const esp_timer_create_args_t args = { .callback = &fsm_timer_cb, .name = "fsm_tick" };
    esp_timer_handle_t timer;
    esp_timer_create(&args, &timer);
    esp_timer_start_periodic(timer, BTN_FSM_TICK_MS * 1000ULL);
}
```

Додаємо в `components/button_fsm/include/button_fsm.h` (обов'язково — інакше `idf.py build` впаде):

```c
void button_fsm_start_timer(void);

// Реалізується у main/app_main.c (Крок 9) — оголошуємо тут, бо викликає її button_fsm.c,
// а не навпаки (той самий "гачок", що й panic_sensor_init у Кроці 6).
void app_fsm_tick_all(uint32_t now_ms);
```

🔴 **ТИПОВА ПОМИЛКА КОМПІЛЯЦІЇ** Якщо пропустити цей заголовок, `idf.py build` впаде на `button_fsm.c` з `error: implicit declaration of function 'app_fsm_tick_all' [-Wimplicit-function-declaration]`. Сучасний toolchain ESP-IDF v5.5.4 (GCC) трактує виклик недекларованої функції як **помилку збірки**, а не попередження. Причина — сам патерн "гачка": `button_fsm.c` викликає функцію, яку **реалізує** `main.c` (Крок 9, на цьому етапі ще не написаний), тож компілятору потрібен лише **прототип**, щоб перевірити виклик, а не сама реалізація.

Перетворення "сирого" АЦП на секунди затримки — новий шматок логіки, `main/app_main.c`:

```c
#define ARM_DELAY_MIN_S  3u
#define ARM_DELAY_MAX_S  15u

static uint32_t arming_delay_seconds(void) {
    uint16_t raw = pot_sensor_get_raw();   // 0..4095, Крок 7 pot_sensor
    return ARM_DELAY_MIN_S + ((uint32_t)raw * (ARM_DELAY_MAX_S - ARM_DELAY_MIN_S)) / 4095u;
}
```

🟣 **РОЗБІР: чому саме так - `pot_sensor` відповідає лише за "прочитати АЦП і кешувати останнє значення"** — той самий принцип, що й у `load_driver`: **інтерпретація числа** (тут — тривалість затримки постановки на охорону) — це вже робота конкретного застосунку (`main.c`), а не компонента. - **`arming_delay_seconds()` — маленька чиста функція** (без побічних ефектів, лише читає `pot_sensor_get_raw()` і повертає число) — легко покрити її окремим тестом і легко зрозуміти без контексту всієї системи (Модуль 2.1: маленькі чисті функції — менше місць для прихованих помилок). - **Чому 3–15 секунд, а не миттєво?** У реальних охоронних панелях затримка постановки (exit delay) дає користувачу час покинути приміщення, не спровокувавши власні датчики руху/дверей. Ми моделюємо це потенціометром замість фіксованого числа — зручно для демонстрації на занятті різних сценаріїв.

🔵 **ЩО СПОСТЕРІГАЄМО** Лог `ESP_LOGI("pot", "затримка = %lu с", arming_delay_seconds());` раз на секунду при обертанні потенціометра плавно змінюється в діапазоні 3–15.

🟠 **МІНІ-ЗАВДАННЯ** Чому в реальних панелях охорони затримку **не** роблять регульованою фізичною ручкою просто на корпусі пульта (на відміну від нашого навчального макета)? Які ризики безпеки це створює? (Підказка: хто ще, крім власника, може дотягнутись до ручки.)

---

09 · Крок 8 · Повна FSM-дебаунс кнопки ⏱ 40 хв

🟢 **META** Реалізувати повну FSM-дебаунс кнопки (Модуль 2.6): чотири стани IDLE/PRESSED/HELD/RELEASE, дебаунс 50 мс на обох переходах (натискання і відпускання), поріг короткий/довгий натиск — 2000 мс, і жодного пропущеного натискання, навіть коротшого за 10 мс.

```c
#include "button_fsm.h"

static inline bool raw_pressed(gpio_num_t pin) {
    return gpio_get_level(pin) == 1;
}

void button_fsm_init(button_fsm_t *btn, gpio_num_t pin) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << pin),
        .mode         = GPIO_MODE_INPUT,
        .pull_down_en = GPIO_PULLDOWN_ENABLE,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&cfg);
    btn->pin = pin; btn->state = BTN_STATE_IDLE;
    btn->state_enter_ms = 0; btn->long_fired = false;
}

button_event_t button_fsm_tick(button_fsm_t *btn, uint32_t now_ms) {
    bool pressed = raw_pressed(btn->pin);
    button_event_t event = BTN_EVENT_NONE;

    switch (btn->state) {
    case BTN_STATE_IDLE:
        if (pressed) { btn->state = BTN_STATE_PRESSED; btn->state_enter_ms = now_ms; }
        break;

    case BTN_STATE_PRESSED:
        if (!pressed) {
            btn->state = BTN_STATE_IDLE;
        } else if (now_ms - btn->state_enter_ms >= BTN_DEBOUNCE_MS) {
            btn->state = BTN_STATE_HELD; btn->state_enter_ms = now_ms; btn->long_fired = false;
        }
        break;

    case BTN_STATE_HELD: {
        uint32_t held_ms = now_ms - btn->state_enter_ms;
        if (!pressed) {
            if (!btn->long_fired) event = BTN_EVENT_SHORT_PRESS;
            btn->state = BTN_STATE_RELEASE; btn->state_enter_ms = now_ms;
        } else if (!btn->long_fired && held_ms >= BTN_LONG_PRESS_MS) {
            event = BTN_EVENT_LONG_PRESS; btn->long_fired = true;
        }
        break;
    }

    case BTN_STATE_RELEASE:
        if (pressed) {
            btn->state = BTN_STATE_HELD;
        } else if (now_ms - btn->state_enter_ms >= BTN_DEBOUNCE_MS) {
            btn->state = BTN_STATE_IDLE;
        }
        break;
    }
    return event;
}
```

🟣 **РОЗБІР: чому саме так - `switch` по `enum`, а не ланцюжок `if/else if`** — компілятор попередить (з `-Wswitch`), якщо додасте новий стан у `enum`, але забудете обробити його тут — легкість розширення, яку обіцяє FSM-підхід. - **Один `state_enter_ms` на всі стани** — під час `PRESSED→HELD` мітка часу **перезаписується**, бо далі рахуємо вже інший інтервал (тривалість утримання, а не дебаунсу). Це економить пам'ять і виключає розсинхронізацію двох незалежних таймерів. - **`PRESSED`: провал сигналу → миттєво назад у `IDLE`** — якщо дребезг "відпустив" пін

раніше 50 мс, стабільного натискання ще не було; наступний фронт HIGH знову підніме нас у `PRESSED` і почне відлік 50 мс заново. Це і є механізм, що **гарантовано ігнорує дребезг**. - **`long_fired` — прапорець одноразовості** — без нього `BTN_EVENT_LONG_PRESS` видавалась би **щоразу** на кожному тіку після 2000 мс, а не один раз. Скидається лише при вході в `HELD` (новий цикл натискання). - **Подія короткого натиску видається у `HELD`, а не в `RELEASE`** — саме в момент `pressed == false` всередині `HELD` ми точно знаємо: дребезг натискання вже пройшов (бо ми тут), і поріг довгого натиску не досягнуто (інакше `long_fired` був би `true`). Це рівно момент фізичного відпускання. - **`RELEASE→HELD` при дребезгу відпускання** — дзеркальна логіка до `PRESSED`: якщо контакт "підстрибнув" назад під час відпускання, стан утримання не втрачається, а просто продовжується. - **10-мс гарантія** — оскільки `button_fsm_tick()` викликається кожні 10 мс апаратним `esp_timer` (Крок 7), максимальна затримка виявлення фронту — 10 мс, і навіть натискання коротше цього інтервалу буде піймано на найближчому тіку.

Новий нюанс, специфічний саме для охоронної логіки (якого не було в попередніх міні-проєктах): - **`BTN_EVENT_SHORT_PRESS` на кнопках ZONE1/ZONE2 означає "симуляція спрацювання датчика"**, а не "перемикач" — тобто це подія, а не toggle-стан. Кожне коротке натискання ZONE-кнопки — це одна незалежна "подія проникнення", що системна FSM (Крок 9) обробляє залежно від того, озброєна система чи ні. - **`BTN_EVENT_LONG_PRESS` на ZONE-кнопках означає "bypass" (виключення зони)** — реальна функція охоронних панелей: якщо один із датчиків несправний або зона тимчасово недоступна (ремонт), її можна виключити з контролю без вимкнення всієї системи.

🔵 **ЩО СПОСТЕРІГАЄМО** На аналізаторі: канал 1 — сирий сигнал GPIO кнопки (дребезг видно, як у Кроці 5), канал 2 — вихід сирени, керований подією FSM. Дребезг на каналі 1 не проходить далі FSM — на виході (канал 2) рівно одна чиста реакція на фізичне натискання.

🟠 **МІНІ-ЗАВДАННЯ** Тимчасово зменшіть `BTN_DEBOUNCE_MS` до 5 мс і подивіться, чи з'являються подвійні спрацювання на вашій конкретній кнопці (порівняйте з дребезгом, виміряним у Кроці 5).

---

10 · Крок 9 · Системна FSM: DISARMED → ARMING → ARMED → ALARM ⏱ 50 хв

🟢 **МЕТА** Побудувати FSM **вищого рівня** над подіями кнопок — головний архітектурний урок цього проєкту: маленькі перевірені автомати (кнопки) стають будівельними блоками для складнішої, обґрунтованої бізнес-логіки (вся охоронна панель).

`main/app_main.c` (повний):

```c
#include "button_fsm.h"
#include "load_driver.h"
#include "pot_sensor.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"
#include "esp_timer.h"

static const char *TAG = "arming_console";

// --- Типи й глобальний стан з Кроку 3 ---
typedef enum { SYS_DISARMED = 0, SYS_ARMING, SYS_ARMED, SYS_ALARM } system_state_t;
typedef struct { const char *name; bool bypassed; bool tripped; } zone_t;

#define N_ZONES 2
static zone_t s_zones[N_ZONES] = {
    { "ZONE1", false, false },
    { "ZONE2", false, false },
};
static volatile bool s_panic_latched = false;
static system_state_t s_sys_state = SYS_DISARMED;
static uint32_t s_state_enter_ms  = 0;
static uint32_t s_arming_total_s  = 0;   // скільки секунд триватиме ARMING (з потенціометра)

#define N_FSM_BUTTONS 3
enum { IDX_ARM = 0, IDX_ZONE1, IDX_ZONE2 };
static button_fsm_t s_buttons[N_FSM_BUTTONS];

#define ARM_DELAY_MIN_S  3u
#define ARM_DELAY_MAX_S  15u
```

```c
static uint32_t arming_delay_seconds(void) {
    uint16_t raw = pot_sensor_get_raw();
    return ARM_DELAY_MIN_S + ((uint32_t)raw * (ARM_DELAY_MAX_S - ARM_DELAY_MIN_S)) / 4095u;
}

// Заявки на "біп" лише СТАВЛЯТЬСЯ в чергу (неблокуюче) з обробників кнопок —
// сам блокуючий vTaskDelay() виконується окремою buzzer_task нижче. ЧОМУ це
// обов'язково саме так, а не прямим викликом load_buzzer_chirp() тут — див.
// 🔴 ПАСТКА одразу після цього блоку коду.
static QueueHandle_t s_chirp_queue;

static void request_chirp(uint32_t ms) {
    xQueueSend(s_chirp_queue, &ms, 0);   // неблокуюче: викликається з esp_timer callback
}

static void buzzer_task(void *arg) {
    uint32_t ms;
    while (1) {
        if (xQueueReceive(s_chirp_queue, &ms, portMAX_DELAY)) {
            load_buzzer_chirp(ms);       // блокуючий vTaskDelay — тут це безпечно, це власна задача
        }
    }
}

static void sys_transition(system_state_t new_state, uint32_t now_ms) {
    ESP_LOGW(TAG, "СИСТЕМА: %d -> %d", s_sys_state, new_state);
    s_sys_state = new_state;
    s_state_enter_ms = now_ms;
}

// --- Обробники подій кнопок FSM (Модуль 2.6 -> бізнес-логіка) ---

static void handle_arm_button(button_event_t ev, uint32_t now_ms) {
    if (ev == BTN_EVENT_SHORT_PRESS) {
        switch (s_sys_state) {
        case SYS_DISARMED:
            s_arming_total_s = arming_delay_seconds();
            sys_transition(SYS_ARMING, now_ms);
            request_chirp(60);
            ESP_LOGI(TAG, "Постановка на охорону: %lu с до ARMED", (unsigned long)s_arming_total_s);
            break;
        case SYS_ARMING:
            sys_transition(SYS_DISARMED, now_ms);   // скасували постановку
            request_chirp(200);
            break;
        case SYS_ARMED:
            sys_transition(SYS_DISARMED, now_ms);
            request_chirp(300);
            ESP_LOGI(TAG, "Знято з охорони.");
            break;
        case SYS_ALARM:
            sys_transition(SYS_DISARMED, now_ms);   // коротке натискання = роззброїти й скинути тривогу
            load_relay_set(false);
            load_buzzer_set(false);
            s_panic_latched = false;
            ESP_LOGW(TAG, "Тривогу скинуто, система роззброєна.");
            break;
        }
    } else if (ev == BTN_EVENT_LONG_PRESS) {
        // Довге утримання ARM у будь-якому стані = аварійний повний ресет
        for (int i = 0; i < N_ZONES; i++) { s_zones[i].bypassed = false; s_zones[i].tripped = false; }
        s_panic_latched = false;
        sys_transition(SYS_DISARMED, now_ms);
        load_relay_set(false); load_buzzer_set(false);
        ESP_LOGW(TAG, "ПОВНИЙ РЕСЕТ ПАНЕЛІ (довге утримання ARM).");
    }
}

static void handle_zone_button(int zone_idx, button_event_t ev) {
    zone_t *z = &s_zones[zone_idx];
    if (ev == BTN_EVENT_SHORT_PRESS) {
        z->tripped = true;
```

```
        ESP_LOGI(TAG, "%s: симуляція спрацювання датчика (bypass=%d)", z->name, z->bypassed);
        if (s_sys_state == SYS_ARMED && !z->bypassed) {
            sys_transition(SYS_ALARM, (uint32_t)(esp_timer_get_time() / 1000ULL));
            ESP_LOGE(TAG, "!!! ТРИВОГА: %s !!!", z->name);
        }
    } else if (ev == BTN_EVENT_LONG_PRESS) {
        z->bypassed = !z->bypassed;
        request_chirp(z->bypassed ? 150 : 50);
        ESP_LOGW(TAG, "%s: bypass %s", z->name, z->bypassed ? "УВІМКНЕНО" : "ВИМКНЕНО");
    }
}

// Викликається кожні 10 мс з esp_timer (Крок 7-8)
void app_fsm_tick_all(uint32_t now_ms) {
    button_event_t ev;

    ev = button_fsm_tick(&s_buttons[IDX_ARM], now_ms);
    if (ev != BTN_EVENT_NONE) handle_arm_button(ev, now_ms);

    ev = button_fsm_tick(&s_buttons[IDX_ZONE1], now_ms);
    if (ev != BTN_EVENT_NONE) handle_zone_button(0, ev);

    ev = button_fsm_tick(&s_buttons[IDX_ZONE2], now_ms);
    if (ev != BTN_EVENT_NONE) handle_zone_button(1, ev);

    // --- Системний перехід ARMING -> ARMED за часом (не подія кнопки, а таймаут) ---
    if (s_sys_state == SYS_ARMING) {
        uint32_t elapsed_s = (now_ms - s_state_enter_ms) / 1000u;
        if (elapsed_s >= s_arming_total_s) {
            sys_transition(SYS_ARMED, now_ms);
            request_chirp(500);
            ESP_LOGW(TAG, "СИСТЕМА ОЗБРОЄНА.");
        }
    }

    // --- PANIC має найвищий пріоритет і "перебиває" систему в БУДЬ-ЯКОМУ стані ---
    if (s_panic_latched && s_sys_state != SYS_ALARM) {
        sys_transition(SYS_ALARM, now_ms);
    }

    // --- Виконавчі дії стану ALARM тримаємо активними, доки не скинуть (handle_arm_button) ---
    if (s_sys_state == SYS_ALARM) {
        load_relay_set(true);
        load_buzzer_set(true);
    }
}

// Реакція на PANIC-переривання (Крок 6) — виконується у panic_task, НЕ в ISR
static void on_panic_deferred(void) {
    s_panic_latched = true;   // сама зміна стану FSM відбудеться в app_fsm_tick_all (наступний тік, ≤10мс)
    ESP_LOGE(TAG, "PANIC зафіксовано, тривога буде піднята на найближчому тіку FSM.");
}

void app_main(void) {
    load_driver_init();
    button_fsm_init(&s_buttons[IDX_ARM],   GPIO_NUM_4);
    button_fsm_init(&s_buttons[IDX_ZONE1], GPIO_NUM_5);
    button_fsm_init(&s_buttons[IDX_ZONE2], GPIO_NUM_6);

    panic_sensor_init(on_panic_deferred);   // Крок 6
    pot_sensor_init();                      // Крок 7

    s_chirp_queue = xQueueCreate(4, sizeof(uint32_t));
    xTaskCreate(buzzer_task, "buzzer_task", 3072, NULL, 4, NULL);

    button_fsm_start_timer();               // Крок 7

    ESP_LOGI(TAG, "Пульт охорони периметра запущено. Стан: DISARMED.");

    while (1) {
        vTaskDelay(pdMS_TO_TICKS(1000));
        ESP_LOGI(TAG, "стан=%d zone1(byp=%d,trip=%d) zone2(byp=%d,trip=%d) pot=%u",
```

```
                s_sys_state, s_zones[0].bypassed, s_zones[0].tripped,
                s_zones[1].bypassed, s_zones[1].tripped, pot_sensor_get_raw());
    }
}
```

🔴 **ПАСТКА: чому `load_buzzer_chirp()` тут НЕ можна викликати напряму**

Найочевидніший варіант — викликати `load_buzzer_chirp(60)` прямо в `handle_arm_button()`, як ми звикли робити із сиреною/зумером у попередніх міні-проєктах. **У цьому коді так робити не можна**, і причина нетривіальна.

`handle_arm_button()` виконується всередині `app_fsm_tick_all()`, яку кожні 10 мс викликає колбек `esp_timer` — `fsm_timer_cb()` (Крок 7). Усі колбеки `esp_timer` в ESP-IDF виконуються в **одній спільній системній задачі** `esp_timer`, і документація прямо забороняє в них блокуватись (`vTaskDelay()`, будь-яке очікування). Якщо це порушити — `load_buzzer_chirp()` заблокує цю спільну задачу на 60–500 мс, поки та одночасно повинна перевстановлювати власні внутрішні періодичні таймери (той-таки 10-мс тік FSM і 50-мс читання потенціометра, Крок 7). Внутрішній зв'язний список таймерів пошкоджується — і результат стовідсотково відтворюваний **при першому ж короткому натисканні ARM**:

```
 Guru Meditation Error: Core 0 panic'ed (LoadProhibited). Exception was unhandled.
Backtrace: ... timer_insert (esp_timer.c:305) ...
```

Лікування — типовий патерн "виробник/споживач": обробник події лише **неблокуюче** кладе заявку в чергу (`request_chirp()`, `xQueueSend(..., 0)` — нульовий таймаут, щоб не заблокуватись, якщо черга раптом переповнена), а сам блокуючий `vTaskDelay()` живе в окремій `buzzer_task`, яка не має жодного стосунку до `esp_timer`. **Загальне правило, яке варто запам'ятати назавжди: колбек `esp_timer` — це завжди "запиши і вийди", ніколи "почекай".** Той самий принцип стосується й колбеків GPIO ISR (Крок 6) — там ми вже застосовували аналогічний прийом (`xQueueSendFromISR` + окрема `panic_task`), просто тут пастка підступніша, бо `esp_timer` — це не переривання, і на перший погляд виглядає як "звичайна задача", у якій начебто можна почекати.

🟣 **РОЗБІР: чому саме так - Системна FSM живе поруч, а не всередині `button_fsm_tick()`** — `app_fsm_tick_all()` спершу викликає всі три кнопкові FSM (Модуль 2.6, незмінна логіка), потім окремо перевіряє **системні** умови переходу (таймаут `ARMING→ARMED`, `PANIC`-прапорець). Це і є "композиція" двох рівнів автоматів: нижній (кнопки) видає **події**, верхній (система) вирішує, що ці події **означають** у поточному контексті. - **`ARMING → ARMED` — перехід за часом, а не за подією кнопки** — на відміну від переходів кнопкової FSM (де тригер — це стан GPIO), тут тригер — сплив лічильника секунд, порахованого від `s_arming_total_s` (взятого з потенціометра в момент входу в `ARMING`, Крок 7). FSM необов'язково реагує лише на зовнішні події — таймаут теж легітимний тригер переходу. - **`s_panic_latched` перевіряється в кожному тіку FSM (раз на 10 мс)** — а не миттєво з ISR-задачі. Це свідомий компроміс: сама детекція натискання PANIC апаратно-миттєва (Крок 6, мікросекунди), але **застосування** цього факту до системного стану відбувається на найближчому тіку (максимум +10 мс) — цього достатньо швидко для будь-якого практичного сценарію і водночас тримає всю логіку зміни стану в одному передбачуваному місці (`app_fsm_tick_all`), а не розкиданою між ISR-задачею й основним циклом. - **`SYS_ALARM` не має "автоматичного" виходу** — єдиний спосіб вийти зі стану тривоги — коротке натискання ARM (свідоме рішення користувача підтвердити, що тривога опрацьована), точно як у реальних панелях, де сирена не замовкає сама. - **`handle_arm_button` — це, по суті, `switch` по `s_sys_state`, вкладений у `switch` по `ev`** — ясний, вичерпний спосіб описати "у кожному з 4 станів коротке натискання ARM означає щось своє". Додати `SYS_MAINTENANCE` (Челендж) означає додати один `case` — компілятор (з `-Wswitch`) підкаже, якщо забудете. - **`s_chirp_queue` + `buzzer_task` розв'язують "швидкий" колбек і "повільну" дію** (див. 🔴 ПАСТКА вище) — обробники подій кнопок лишаються швидкими й передбачуваними (жодного `vTaskDelay` у виклику з `esp_timer`), а вся робота з реальним звучанням зумера — в одному, окремому, легко протестованому місці.

Оновлена діаграма системних переходів:



🔊 **ДОВІДКА: коли лунає звук, а коли ні** Не кожна подія кнопки супроводжується "біп" — це свідомий вибір у Кроці 9, а не недогляд. Плутанина тут поширена (особливо коли тестуєте систему кнопка за кнопкою), тож зведено в одну таблицю:

Кнопка / подія	Перехід стану	Звук
ARM коротко	DISARMED → ARMING	`request_chirp(60)`
ARM коротко	ARMING → DISARMED (скасування)	`request_chirp(200)`
ARM коротко	ARMED → DISARMED (зняття)	`request_chirp(300)`
ARM коротко	**ALARM → DISARMED (скидання тривоги)**	**тиша** — сирена просто вимикається (`load_buzzer_set(false)`), без окремого підтверджувального “біп”
ARM довго (≥2с)	будь-який стан → повний ресет	**тиша** — так само, лише вимкнення, без “біп”
ZONE1/ZONE2 коротко	DISARMED, або ARMED+bypass	тиша, лише лог `trip=1`
ZONE1/ZONE2 коротко	ARMED і НЕ bypass	тривога: **безперервний** гул (`load_buzzer_set(true)` виконується щотіку в `app_fsm_tick_all` , доки не роззброять)
ZONE1/ZONE2 довго (≥2с)	перемкнути bypass	`request_chirp(150)` при увімкненні / `request_chirp(50)` при вимкненні
PANIC (переривання)	будь-який стан → ALARM	безперервний гул, той самий механізм, що й тривога від зони

(`request_chirp()` лише ставить заявку в чергу — сам звук програє `buzzer_task` з тією ж тривалістю в мілісекундах, див. 🔴 ПАСТКА вище.)

Отже, якщо після натискання **однієї конкретної** кнопки немає звуку — спочатку перевірте, чи це не одна з двох “тихих за задумом” гілок вище (скидання ALARM коротким ARM, або повний ресет довгим ARM), перш ніж підозрювати апаратну несправність. Обидві ці гілки не викликають `request_chirp()` взагалі — лише `load_buzzer_set(false)` , тобто якщо сирена вже й так мовчала, натискання нічим не підтвердиться на слух.

🎚️ **ДОВІДКА: що саме змінює потенціометр** У поточному `main.c` потенціометр **не** керує темпом писку напряму — Крок 10 (`arming_beep_task` , наростаючий писк під час відліку постановки) — це окрема, додаткова задача, і без неї потенціометр впливає лише на **одне число**: `arming_delay_seconds()` (Крок 7). Це число зчитується **рівно один раз** — у момент короткого натискання ARM зі стану `DISARMED` (`s_arming_total_s = arming_delay_seconds();`) — і “заморожується” на весь цей цикл постановки. Звідси три практичні наслідки: - крутити ручку **під час** `SYS_ARMING` — на вже запущений відлік не вплине; - крутити ручку в `DISARMED` / `ARMED` / `ALARM` — вплине лише на **наступну** спробу постановки на охорону; - діапазон — 3–15 с (`ARM_DELAY_MIN_S` / `ARM_DELAY_MAX_S`), лінійна мапа сирого АЦП (0..4095, `pot_sensor` , Крок 7).

Побачити ефект можна двома способами: лог `[POT=...]` (сире значення АЦП, раз на секунду, у фоновому `while(1)` з `app_main`) і рядок `"Installation of security: %lu s to ARMED"` , який друкується в момент входу в `SYS_ARMING` — саме там видно вже переведене в секунди значення, яке “заморозилось” для цього циклу.

🔵 **ЩО СПОСТЕРІГАЄМО** Коротко натискаємо ARM → `SYS_ARMING` , у логах — зворотний відлік по секундах (додайте лог у `app_fsm_tick_all` , Челендж). Після спливу — `SYS_ARMED` , довгий “біп”-підтвердження. Тепер коротке натискання ZONE1 — миттєва `SYS_ALARM` , сирена й реле активні, доки не натиснете ARM коротко.

🟠 **МІНІ-ЗАВДАННЯ** У стані `SYS_ARMED` , якщо зона в `bypassed` , спрацювання **ігнорується** повністю (навіть `tripped` не рахуємо в лог як тривожне). Чи це правильна поведінка для реальної системи, чи варто хоча б **логувати** “проігноровано через bypass” для подальшого аудиту? Додайте таке логування.

---

11 · Крок 10 · Потенціометр → затримка й прискорення писку ⏱ 20 хв

🟢 **META** Додати реалістичний штрих: під час `SYS_ARMING` зумер попискує, і темп писку **прискорюється** в міру наближення кінця відліку — стандартна поведінка реальних охоронних панелей (спокійний писк → частий → безперервний сигнал “час вийти закінчився”).

```
// окрема фонова задача (не esp_timer), бо темп писку тут змінюється щоразу
static void arming_beep_task(void *arg) {
    while (1) {
        if (s_sys_state == SYS_ARMING) {
            uint32_t now_ms = (uint32_t)(esp_timer_get_time() / 1000ULL);
            uint32_t elapsed_s   = (now_ms - s_state_enter_ms) / 1000u;
            uint32_t remaining_s = (elapsed_s < s_arming_total_s) ? (s_arming_total_s - elapsed_s) : 0;

            // чим менше залишилось часу, тим частіший писк: 500 мс (спокійно) .. 80 мс (терміново)
            uint32_t beep_period_ms = 80 + (remaining_s * 420) / (ARM_DELAY_MAX_S);

            load_buzzer_set(true);
            vTaskDelay(pdMS_TO_TICKS(30));     // короткий "тік", а не довгий сигнал
            load_buzzer_set(false);
            vTaskDelay(pdMS_TO_TICKS(beep_period_ms));
        } else {
            vTaskDelay(pdMS_TO_TICKS(100));    // не ARMING — просто чекаємо, нічого не робимо
        }
    }
}
// у app_main(): xTaskCreate(arming_beep_task, "arming_beep", 3072, NULL, 4, NULL);
```

🔴 **ПАСТКА: чому стек 3072, а не “стандартні” 2048**

`load_buzzer_set()` і `load_relay_set()` (Крок 4) всередині викликають `ESP_LOGI` / `ESP_LOGW` — форматований вивід у лог (`vprintf` -подібний код) сам по собі використовує помітну частину стека задачі, яка його викликає. Якщо дати `arming_beep_task` (і `buzzer_task` вище) лише 2048 байт, перший же вхід у `SYS_ARMING` — тобто перший же виклик `load_buzzer_set(true)` з цієї задачі — впаде з:

```
***ERROR*** A stack overflow in task arming_beep_task has been detected.
```

Це друга, окрема від 🔴 ПАСТКИ вище, причина краху — обидві виявляються лише тоді, коли реально доходить до `SYS_ARMING`, тож на побіжному тестуванні “чи компілюється і чи блимає” жодна з них не проявиться. Загальне правило: для будь-якої задачі, що викликає `ESP_LOGx` і/або GPIO/ADC-драйвери, не економте на стеку — 3072 байт як стартова точка помітно безпечніше за “мінімальні” 2048.

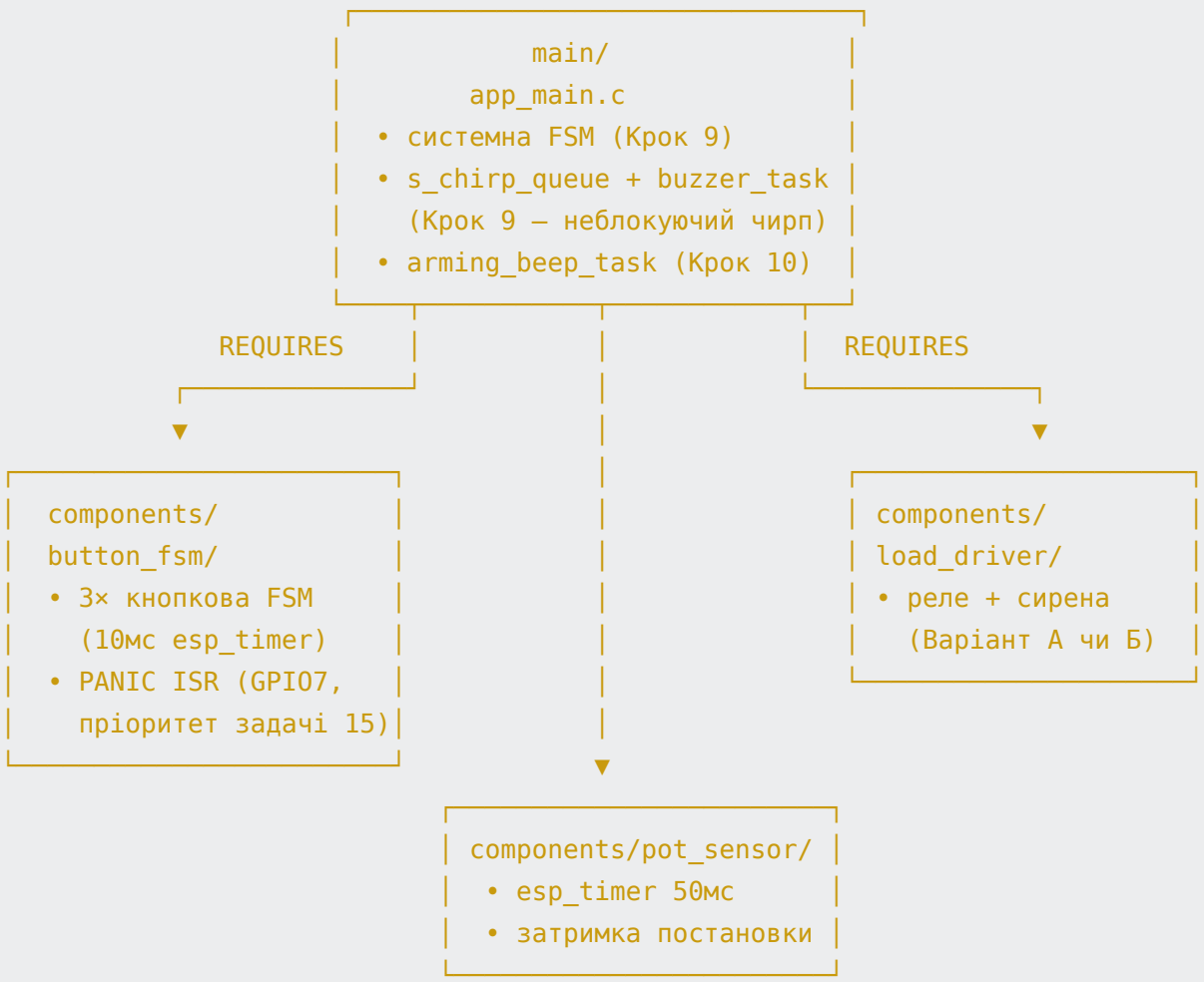
🟣 **РОЗБІР: чому саме так - `remaining_s`, а не `elapsed_s`, керує темпом** — писк повинен **прискорюватись** із наближенням кінця, тобто менше часу залишилось → менший `beep_period_ms`. Формула `80 + remaining_s * 420 / MAX_S` лінійно мапить `remaining_s` від `ARM_DELAY_MAX_S` (повільно, 500 мс) до `0` (швидко, 80 мс). - **Окрема FreeRTOS-задача, а не третій `esp_timer`** — період тут **сам змінюється** щоразу заново залежно від `remaining_s`, простіше й наочніше виразити це через `while(1) + vTaskDelay(period_ms)` у власній задачі, ніж постійно перестворювати `esp_timer` з новим періодом. Обидва підходи в ESP-IDF правильні; вибір залежить від того, наскільки часто змінюється період. - **`else`-гілка з `vTaskDelay(100)`** — коли ми не в `ARMING`, задача не повинна “крутитись” вхолосту на максимальній швидкості (це б даремно вантажило CPU) — 100 мс цілком достатньо для реакції, коли стан зміниться на `ARMING` знову.

🔵 **ЩО СПОСТЕРІГАЄМО** Постановка на охорону з великою затримкою (потенціометр на максимум, ~15 с) — писк спочатку рідкий, до кінця відліку стає помітно частішим.

🟠 **МІНІ-ЗАВДАННЯ** Зробіть так, щоб останні 3 секунди перед `ARMED` писк був **безперервним** (не тіками, а суцільним сигналом) — де саме у формулі/умові ви це додасте?

---

12 · Крок 11 · Фінальна архітектура ⏱ 15 хв



🟣 **РОЗБІР: чому саме так** Жодна стрілка не йде між `button_fsm` і `load_driver` напряму — вони не `REQUIRES` одне одного (перевірте `CMakeLists.txt`, Крок 2). Це і є найважливіший висновок цього проєкту: **хороша модульна архітектура ESP-IDF відокремлює перевикористовувані апаратні драйвери (`load_driver`, `pot_sensor` — жодного слова “arm”/“zone”/“panic” всередині) від логіки конкретного продукту, яка вся живе в `main/app_main.c`.**

🟠 **МІНІ-ЗАВДАННЯ** Домалюйте на цій схемі стрілку `arming_beep_task` (Крок 10) — з чим вона взаємодіє? (Підказка: читає `s_sys_state` / `s_arming_total_s` і викликає `load_buzzer_set()` — тобто це, по суті, ще один “клей”, як і `app_main`, тільки для одного вузького сценарію.)

---

13 · Крок 12 · Логи й діагностика ⏱ 15 хв

🟢 **МЕТА** Використати `ESP_LOGx` замість голого `printf` — стандартна практика ESP-IDF з рівнями важливості й тегами компонентів (Модуль 2.1: чистий, підтримуваний код).

🟣 **РОЗБІР: чому саме так** У системі безпеки особливо важливо розділяти рівні: `ESP_LOGE` — лише для `SYS_ALARM`/PANIC (те, що оператор повинен побачити миттєво навіть у “зашумленому” логу), `ESP_LOGW` — зміни системного стану, `ESP_LOGI` — рутинний статус.

🔵 **ЩО СПОСТЕРІГАЄМО** У кольоровому виводі `idf.py monitor` тривога (`ESP_LOGE`, червоний) одразу впадає в очі серед звичайних `ESP_LOGI` (зелений) рядків статусу.

🟠 **МІНІ-ЗАВДАННЯ** Додайте `ESP_LOGD` з повним дампом `s_zones[]` у кожному тіку та порівняйте об’єм логів при увімкненому/вимкненому Debug-рівні за 10 секунд роботи.

13.1 · Просте тестування: 4 кнопки й ручка

На платі — 4 кнопки й одна ручка (потенціометр). Ось що натискати і що має статись.

Ручка (потенціометр) - Крутиш в будь-який момент → **звуку немає ніколи**, це нормально. Вона просто задає, скільки секунд триватиме “підготовка до охорони” (наступний пункт).

Кнопка 1 — ARM (постановка/зняття з охорони) - Натиснув коротко (система вимкнена) → **короткий біп**, почалась підготовка (кілька секунд, скільки — залежить від ручки). - Почекав кілька секунд, нічого не натискав → **другий, довший біп** — система тепер “на охороні”. - Натиснув коротко (поки на охороні) → **біп** — система знята з охорони. - **Утримав 2 секунди** (в будь-який момент) → **звуку НЕ буде**. Все скидається мовчки — це нормально, так і задумано.

Кнопки 2 і 3 — ZONE1, ZONE2 (кнопки-датчики) - Система ще НЕ на охороні → натиснув коротко → **звуку немає**, просто “клік” у лог, нічого більше. - Система НА охороні → натиснув коротко → починається **гучний безперервний звук** — це тривога. - Щоб зупинити тривогу — натисни **кнопку 1 (ARM) коротко**. Звук зникне одразу, **без жодного біп** — це та сама “тиха” поведінка, що й вище, це нормально, не поломка. - Утримав кнопку 2 секунди → **короткий біп** — ця зона тепер ігнорується (можна перевірити: постав на охорону і натисни цю ж кнопку — тривоги вже не буде).

Кнопка 4 — PANIC (тривога) - Натиснув коротко, в БУДЬ-який момент (навіть якщо система вимкнена) → одразу **гучний безперервний звук**. - Зупиняється так само — коротко кнопкою 1 (ARM), теж без біп.

Підсумок одним реченням: звук є майже завжди, коли щось *вмикаєш* чи *підтверджуєш* — і його свідомо немає лише у двох випадках, коли кнопкою 1 щось *вимикаєш/скидаєш* (зняти тривогу, повний ресет). Якщо звуку немає десь в іншому місці зі списку вище — тоді це вже несправність, а не задум.

---

14 · Челенджі ⏱ ~1 год

Челендж 1 · Duress-код (тихий сигнал під примусом) ⏱ 20 хв

Реалізуйте: якщо користувача змушують роззброїти систему, довге утримання ARM (замість короткого) у стані `SYS_ARMED` роззброює **видимо** (сирена й реле вимикаються, як завжди), але одночасно **тихо** встановлює окремий прапорець `s_duress` і, наприклад, залишає зумер вимкненим, тоді як реле продовжує коротко клацати раз на кілька секунд (немов “усе normal”, але прихована сигналізація активна). Поясніть у коментарі, чому такий сигнал ніколи не повинен бути чутним.

Челендж 2 · Третя й четверта зона ⏱ 15 хв

Збільште `N_ZONES` до 4, повторно використавши ту саму кнопку `BTN2` / `BTN3` для “подвійної” зони через довге утримання під час постановки (Челендж-розширення) — або обґрунтуйте, чому для 4 повноцінних зон фізично бракує кнопок у наборі (тільки 4 кнопки, одна вже зайнята ARM, одна — PANIC).

Челендж 3 · Лічильник хибних тривог і lockout ⏱ 15 хв

Якщо `SYS_ALARM` спрацьовує 3 рази поспіль протягом 60 секунд без проміжного `SYS_ARMED`-періоду довшого за 10 с — це, ймовірно, несправний датчик. Заблокуйте відповідну зону (`bypassed = true`) автоматично й залогуйте `ESP_LOGE` .

Челендж 4 · Індикація стану без LED ⏱ 10 хв

У наборі немає LED — вигадайте, як передати “статус ARMED” лише через зумер (наприклад, один короткий “біп” раз на 5 секунд, поки система озброєна, у фоновій задачі) так, щоб це не заважало сприймати сигнал `SYS_ALARM` .

Челендж 5 · Збереження стану bypass у NVS ⏱ 20 хв

Використайте `nvs_flash` (вбудований у ESP-IDF), щоб `bypassed`-статус зон переживав перезавантаження плати (реалістична вимога: панель не повинна “забувати” налаштування обслуговування після втрати живлення).

✅ ПІДСУМОК: ЩО МАЄТЕ ОТРИМАТИ В КІНЦІ

- ☐ Повністю самостійний ESP-IDF-проєкт (`arming_console`) з правильною модульною архітектурою: окремі **компоненти** (`components/…`), а не один файл `main.c` .
- ☐ `load_driver` — драйвер реле й сирени у двох варіантах підключення (клеми реле / транзистор+PWM), що перемикається одним макросом і не вимагає зміни жодного виклику в `main.c` .
- ☐ 3 кнопки (ARM, ZONE1, ZONE2) із повною FSM-дебаунс логікою (Модуль 2.6).
- ☐ Кнопка PANIC на **справжньому апаратному перериванні** з найвищим пріоритетом задачі в системі — і чітке розуміння, чому критичний сигнал безпеки не можна ставити в чергу опитування нарівні зі звичайними кнопками.
- ☐ Робоча **дворівнева FSM**: кнопкові автомати (низький рівень, Модуль 2.6) і системний автомат `DISARMED→ARMING→ARMED→ALARM` (високий рівень, бізнес-логіка), з чітким розумінням, чому їх не можна змішувати в один компонент.
- ☐ Постановка на охорону з регульованою потенціометром затримкою й реалістичним прискоренням писку зумера при наближенні кінця відліку.
- ☐ Bypass-логіка зон і повний ресет системи через довге утримання ARM.
- ☐ Чітке розуміння, чому колбек `esp_timer` не можна блокувати (Крок 9, 🔴 ПАСТКА) і чому задачі, які викликають `ESP_LOGx` /GPIO-драйвери, потребують запасу стека (Крок 10, 🔴 ПАСТКА) — обидві помилки компілюються без жодного попередження й падають лише в рантаймі.
- ☐ Ви можете пояснити словами різницю між “подією” (коротке натискання ZONE — одноразова симуляція спрацювання) і “станом-перемикачем” (ARM — toggle), і чому вибір саме такої семантики для кожної кнопки логічний у контексті охоронної системи.